Making A New Quest

A complete Quest definition in Majesty contains several parts.

- A Quest Definition file (.mqxml)
  - This is the master file for a Quest. It contains a basic description and display name for the Quest and references to the other files the Quest uses, such as the Template, GPL Bytecode, etc. These files are defined below.
- A Quest Template (.q)
  - This file contains the layout of the map and its starting conditions. It's contents are edited by the RGSEditor application.
- GPL Bytecode (.bcd)
  - This is the compiled GPL (Game Play Language) source code that the Quest will use to run.
- Game Object Definitions (.xml)
  - These are optional definitions that create or modify existing game objects, such as Buildings, Characters, or Sounds.
- Art/Sound Binary Data (.cam)
  - These files contain the images for the game objects as well as their sounds.
- RGS Constants (.rgs)
  - These are definitions for various Random Generation System data that is used to generate the map. The game comes with many pre-defined constants that a quest can use. The Quest can also define new ones or modify existing ones.

Adding new Game Object Definitions, Binary Data and changing RGS Constants is optional, a Quest can be created using just the existing assets of Majesty.

Getting Started

To make a new Quest, the first thing to do is to create the Quest Definition file and the Quest Template. These files are created and edited with the RGSEditor (Random Generation System) found in the SDK folder.

Random Generation System (RGS) Editor

Launch the RGSEditor and under the File... menu, select New Quest..., which will bring up the Create New Quest dialog.

In this dialog, you define define the following:

- The name of the GPL function to call on initialization of the Quest. This function name

is not case sensitive, and must not contain spaces or other space characters other than the underscore (_).

- The data set the Quest will use as a starting point, this can be the Original Majesty data set or the Northern Expansion data set.
- The size of the map that the Quest will generate.
- A random seed for the random number generator. If this value is 0, then a new seed will be used every time the Quest is started, else it will use the fixed one provided. Using a fixed seed is often helpful during the creation of Quests as it allows the developer to generate the same map when testing the Quest.

For our example, we will keep “InitializeMyQuest” as the GPL entry point function name and we will use the “Majesty:Original” as the source data set. Press OK to exit the Quest definition dialog.

After creating the Quest definition, there are still a few things to add before you will see a generated map. The first of which is to add at least one Unit Pattern. To do this, select Edit Unit Patterns... from the Quest menu.

Unit Patterns

A Unit Pattern describes the placement layout of one or more individual units. The layout is a 5 by 5 grid, with a variable resolution of the grid that adjusts the spacing between center points of the cells of the grid. The spacing is also adjusted based on the size of the map. If the spacing would cause a unit to be placed on top of another unit, the unit is placed as close as possible, without overlapping. One or more units can be selected to possibly be placed at a grid location. Each unit definition is placed only once for the entire pattern. The placement grid is also randomly rotated to add more variety to the placement of the pattern.

To continue with the example, press the New... button in the Pattern section of the dialog. A dialog will pop up requesting a name for the pattern, in this case we will be creating the Unit Pattern that defines the starting location for the player. Enter “Player Start” as the name of the pattern and press OK.

The next step is to add one or more Unit instances to the pattern layout.

Units

Each pattern defines one or more units to place. A unit represents a building, a character, a monster, or landmark object. Each defined unit will be placed only once. To allow the same type of unit to be placed in the pattern more than once, the pattern allows multiple entries that have the same unit type.

Press the Add... button in the Unit section. This will bring up a dialog where you can select a category filter, then the unit type in the category. In this case we want to add a Palace for the player. Select “Buildings” in the category drop down, then “Palace” from the Unit Types drop down, then select OK.

The next step is to place the Palace in the Layout Grid.

Layout Grid

The Layout Grid defines where each Unit can possibly go in the pattern. When the Random Generation System places the Unit Pattern on the map, each unit definition will be placed only once. Marking the same Unit instance at multiple locations in the Pattern's Layout Grid is used to allow for randomizing the appearance of the placement, not for placing the same type of unit in multiple locations in the pattern. To place the same unit type in multiple locations, you would make additional Unit instances that reference the same unit type.

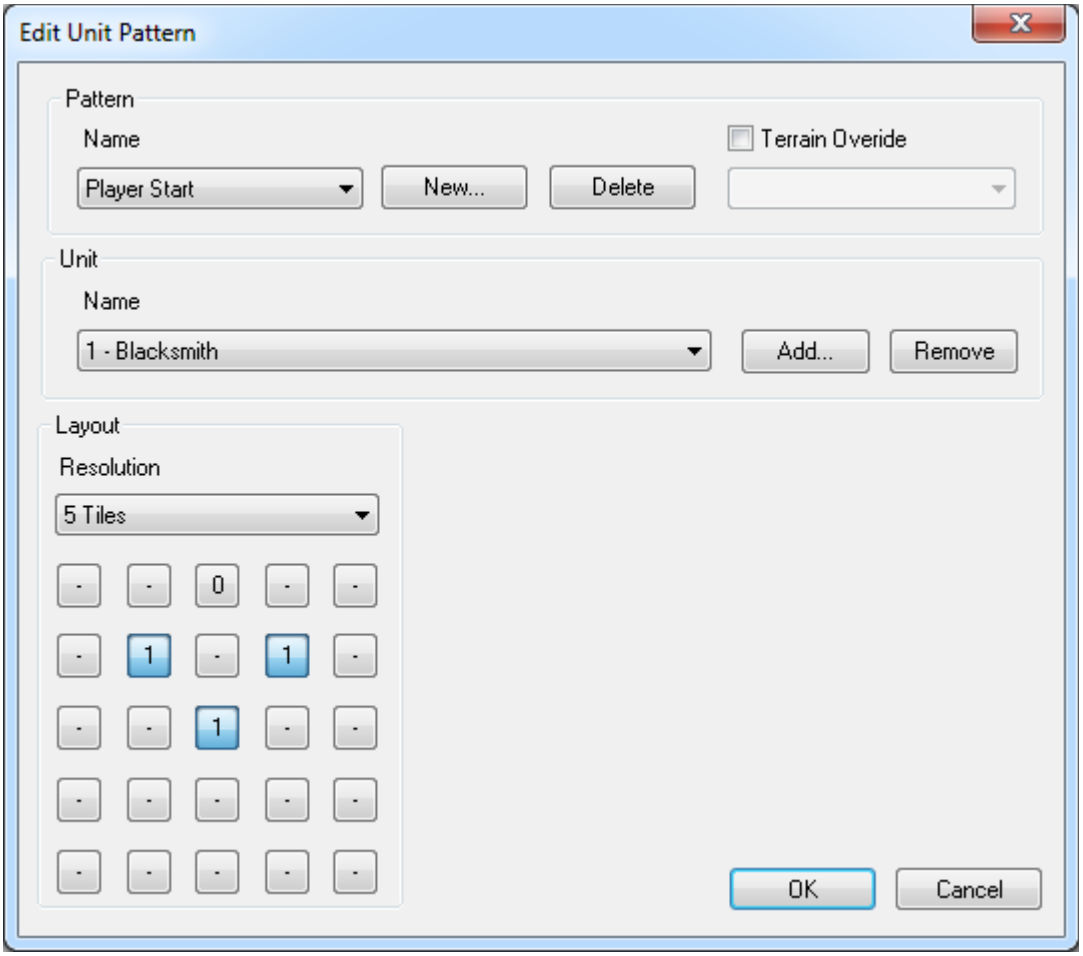
More that one unit definition can be set at a grid location. In that case, the grid entry will show an '*'. When the pattern is generated, only one unit will be chosen for the location. To add a Unit instance to a placement location, simply click on the layout location's button. If the Unit instance is already at the selected location, the Unit will be removed from that location.

The Layout Grid also has a control for the spacing of the Units in the Grid, using the Resolution drop down. This determines the distance between the center of a layout grid entry to its neighbor. A tile is 32 pixels wide so a resolution of 3 means that each layout item is placed 96 pixels from its neighbor. The placement code will not allow objects to overlap, so a minimum spacing will be enforced if the placement would cause an overlap.

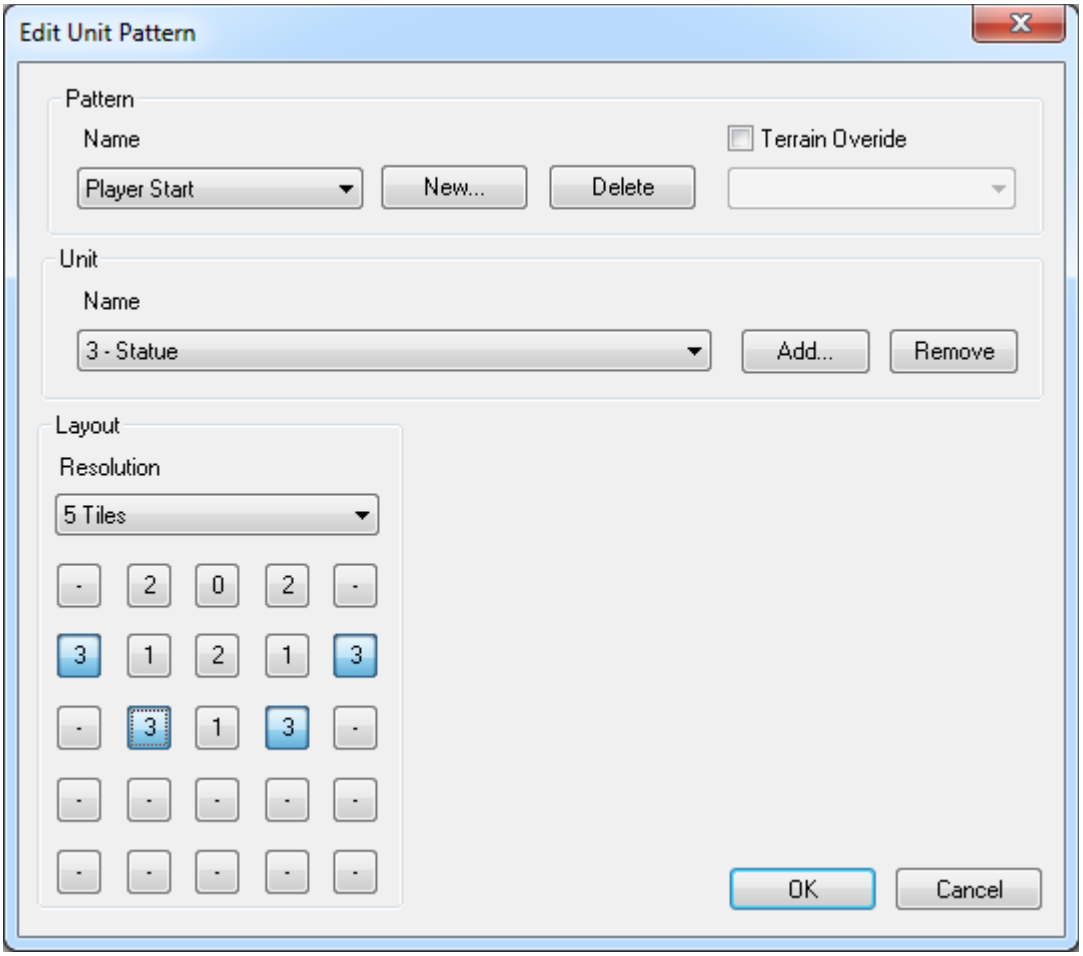
To provide for a some additional randomization of placement, when units that can move (monsters, heroes) and terrain objects are placed, they are shifted by a random amount of up to 100 pixels, which equates to about 3 tiles.

Continuing with the example, let's place the Palace at the top, center Layout grid. To place it, click on the top center grid entry. A 0 will appear at that location and the grid entry will also be highlighted denoting that the grid entry contains a placement for the currently selected Unit. Let's give the player a little head start by also giving them a Blacksmith. Click Add... in the Unit section, then select the “Buildings” category, then select “Blacksmith” and press OK.

We will now select where the Blacksmith will be placed, but let's allow the Blacksmith to appear in a few different locations. Click on grid locations, 3,3, and 2, 2 and 4, 2, so the Layout grid looks like this.



The Blacksmith will be randomly placed at **one** of the three locations specified in the layout. As an example of how to add more than one of the same type of unit to the map, select Add... in the Unit section, and select the “Buildings” category and select “Statue” from the Unit Type list, then press OK. Now repeat the process of adding a Statue. Your Unit list will now have one entry named “2 – Statue” and “3 – Statue”. Select the “2 – Statue” entry and click on some layout locations, then select “3 – Statue” and select some different locations. Making the layout look like this:



When the Unit Pattern is placed, there will be at least two Statues, one of which will be placed where a 2 appears and one where a 3 appears.

We are done creating the “Player Start” Unit Pattern. To add it to the template, press OK.

The next step is to create a Force Pattern that will place the Unit Pattern on the map.

Force Patterns

A Force Pattern defines how each of the Unit Patterns that have been defined are placed on the map as a whole. Most quests will only have one Force Pattern which lays out all the Unit Patterns for the Quest.

Continuing with the example, Select Edit Force Patterns... from the Quest menu to open up the Force Pattern editor. In the Force Pattern editor, press New... to create a new Force Pattern and in the name dialog, type “Quest Layout”.

We now want to add an instance of the “Player Start” Unit Pattern we created in the previous steps. Press the Add... button in the Unit Patterns section, select Player Start and press OK. Now we will select a possible location for the Unit Pattern instance. Select the top, left Map Layout entry. We have enough information in our Force Pattern. Press OK to exit the Force

Pattern editor.
The next step is to define what the map's landscape looks like.

Region Patterns

A Region Pattern defines what the map's terrain looks like as well as what landscape objects, such as trees and rocks, are placed on the terrain.

Continuing with the example, Select Edit Region Pattern... from the Quest menu to open up the Region Pattern editor. In the Region Pattern editor, select New... in the pattern section and set the name to “Quest Terrain”.

The next step is to give the Region Pattern one or more Region Patches that will define the look of the Region.

Region Patches

A Region Patch contains reference to a Terrain Pattern, Fractal Settings and a Landscape Pattern.

Select Edit Region Patches... in the Region Editor to open the Region Patch editor. In the Region Patch editor, select New... and name the Patch “Grass”. The next step is to define the Terrain Pattern, Fractal Setting and Landscape Pattern for the Region Patch.

Terrain Pattern

The Terrain Pattern controls what type of terrain texture will be used. There are six basic types of textures the terrain can use and some of those types have a 'child' type, which is a slightly different type of the same texture. The basic textures are:

Dirt	– Brown dirt, with occasional small rocks and tufts of grass.
Plains	– Light green grass.
Plains, child	– Dark green grass
Arid	– Very light green grass with occasional tufts of brown grass.
Arid, child	– Yellowed grass, with the occasional tuft of light green grass.
Scorch	– Gray dirt with the occasional tuft of grass.
Scorch, child	– Dark gray dirt with the occasional rock.
Swamp	– Green/blue dirt with the occasional tuft of dark grass.
Swamp, child	– Red/Brown dirt with the occasional tuft of dark grass.
Snow	– Snow covered dirt.

The Terrain Pattern drop down in the Region Patch editor will contain all the pre-defined Terrain Patterns from the constant set. The names of the Terrain Patterns generally define what the pattern contains. See the Editing Constants section for information on adding/editing the Terrain Patterns.

Continuing with the example, from the Terrain Pattern drop down, select the #Plains_Mostly_Child entry. This pattern contains the grassy plains texture, and when placed, will use mostly the child version of the texture, which is dark green.

Fractal Settings

The fractal settings for a patch control how bumpy the terrain in the patch will be. There are several predefined fractal definitions in the constant set. See the Editing Constants section for information on adding/editing the fractal definitions.

For the example, let's use the Rolling Hills fractal setting.

Landscape Pattern

A Landscape Pattern defines what types of landscape objects are placed in the Region Pattern as well as how many. The constants set has many pre-defined Landscape patterns, some of which are defined specifically for one of the original Quests. You can use any of the available patterns or create a new one. See the Editing Constants section for information on adding/editing the Landscape Patterns.

For our example, we will use a pre-defined pattern. In the Landscape drop down, select #BBC_Grass.

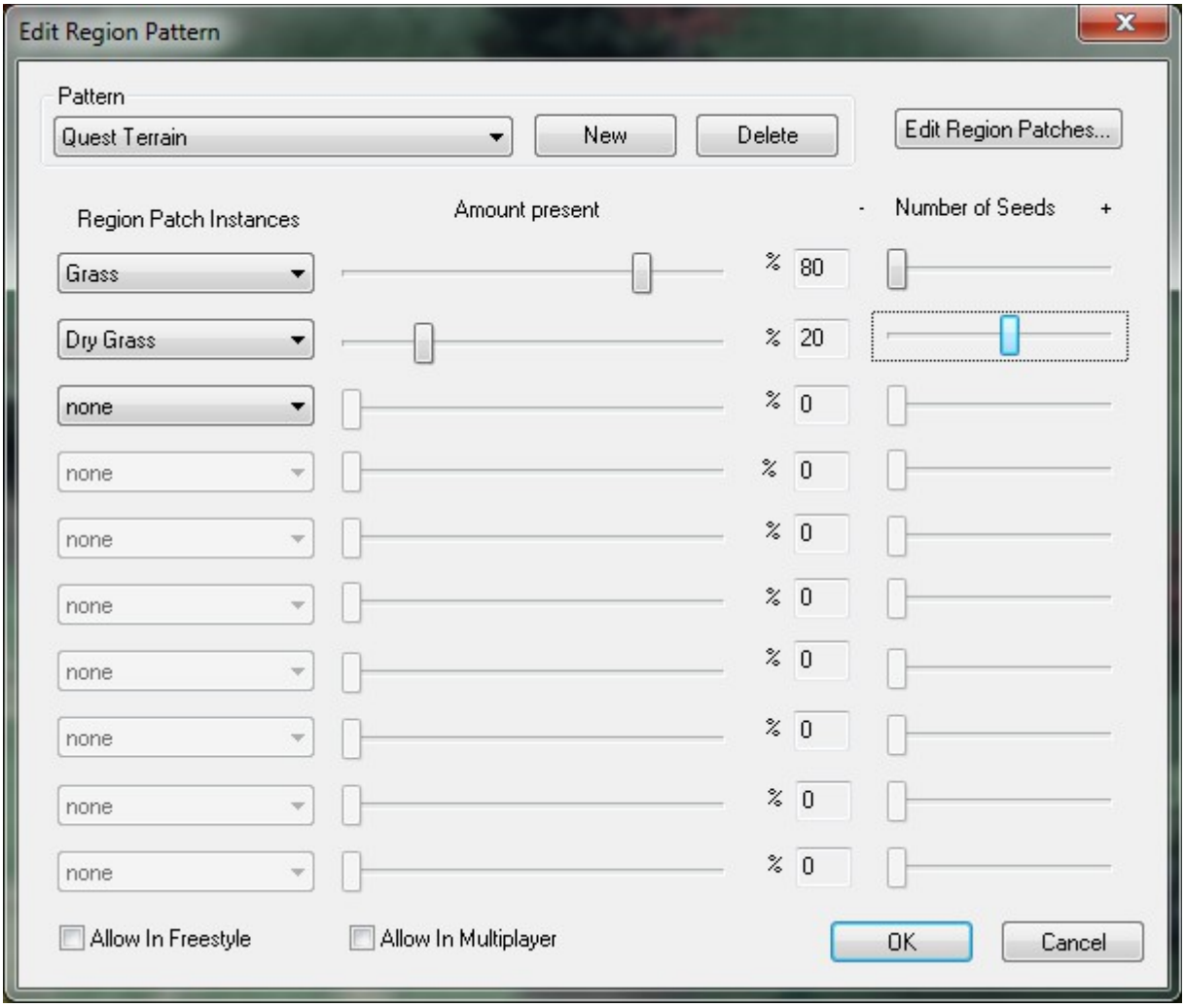
The is the Landscape pattern from The Bell Book and Candle Quest, which places green, leafy trees and some rocks on the terrain.

Press **OK** to add the Region Patch and return to the Region Pattern dialog.

New we can add the Grass Region Patch we just made to the Region Pattern. Select the first drop down in the Region Patch Instances column and select the “Grass” Region Patch.

Let's give a little more variety to the terrain and create another Region Patch type.

Click on Edit Region Patches... and in the Region Patch editor, press the New... button and type “Dry Grass” for the name. For the Terrain Pattern, select #Arrid_Mostly_Child, which will use the lighter, dry grass texture. For the Fractal Settings, choose Small Hills and for the Landscape Pattern, choose #Fertile_Mixed. Press OK to return to the Region Pattern editor and select the Dry Grass Region Patch we just created from the second Region Patch Instances dropdown. We can now determine how much of this new terrain to have on the map by moving the Amount Present slider. Slide it up to around 20 percent. We can also adjust how the region patch values are distributed across the map, by selecting the number of seed points the Random Generator will use when placing the pattern. Slide the Number of Seeds slider about half way. The editor should look like this:



Select the OK button. The Random Generation System now has enough data to generate a map. You should see something like this.

Since the map is randomly generated, your map will not look exactly the same and you may not even see you Palace. If you don't you might be wondering why because we did place the “Player Start” Unit Pattern in the upper left corner of the layout grid. The reason you may not see your Palace is because to aid in the randomization of the maps, the layout is randomly rotated as well. Use the scroll bars on the main view to scroll to the four corners of your generated map and you will find your Palace in one of them.

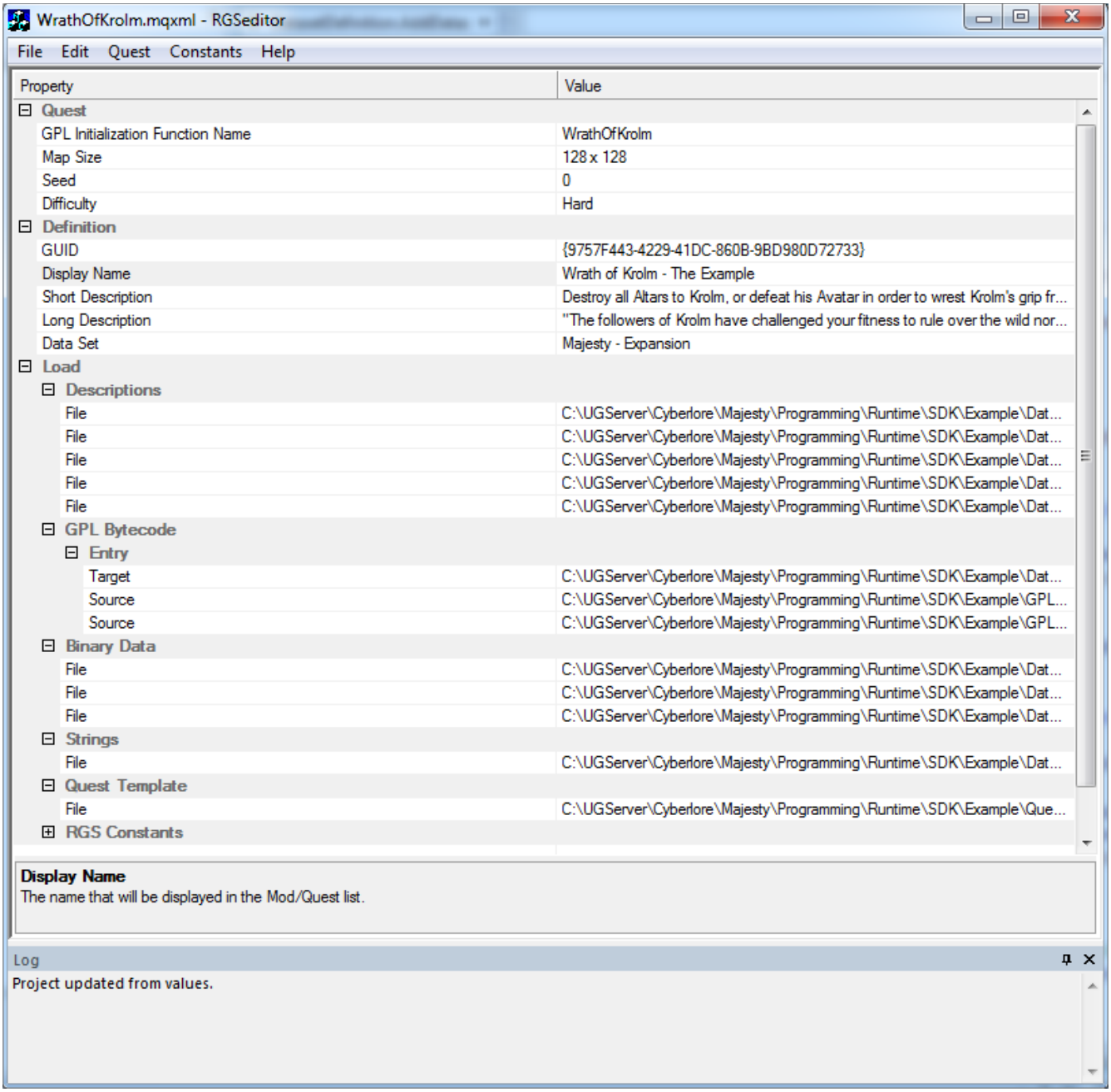
More to come...

The Project View

The files and data value of the Quest Definition can be viewed by the Project View of the RGSEditor. The view can be changed from the Map View to the Project View, by selecting Project View from the Edit menu.

The Project View allows for changing things like the Quest's description, its difficulty, and the files that the project references.

Below is the Project View of the Wrath Of Krolm example. The definition file format is also explained in the Quest Definition File Format section.



At the top are the various Quest values, such as the GPL initialization function name, the map size, the random seed and the difficulty. The fields can be editing by clicking in the Value column.

The **GUID** field in the Definition section is the ID that was assigned when the Quest/Mod was created. Changing it will make the Quest/Mod be associated with a different ID, and may render save games that used the Quest/Mod unusable. However, if you are using a previous Quest/Mod as a template to start a new Quest/Mod from, you MUST change the ID. You can generate a new ID by clicking on the current ID to select it, then press the ellipses button (…) that will appear at the right end of the ID field.

The short and long description fields are used when displaying information about the Quest/Mod in the game. If you need a line break in the text, use the tag [newline] to signify where to add the line break.

The base data set the Quest/Mod uses can be adjusted in the Data Set pulldown. For Mods, there will be an extra option of Any if the Mod can use any data set.

The Load section of the definition describes what the Quest/Mod will load. Right clicking on a sub section will bring up a context menu that will allow you to add/remove files from that section. The GPL Bytecode sub-section is special in that you specify the bytecode data to load, which will add a further sub-entry, which you can then right click on to add the source files that make up the bytecode.

When adding a file to any sub-section, the file does not need to exist. This is especially handy when adding in the bytecode file, because it will not necessarily exist until you compile the source code. If you need to add a file that does not yet exist, browse to the folder the file will be in, in the file search dialog presented when you select Add... from the context menu, then manually type in the file name. The file will not be created, but the reference to the file will be added.

When adding files to the project, you will note that the file paths are fully qualified. That is, they have a complete path to the file. These will automatically be made into paths that are relative to where the Quest/Mod is saved. It is best practice to have all the files that a Quest/Mod uses be at or below the location of where you are saving the project. If the files are somewhere else on your computer, the game may be unable to properly locate the files, and/or the Quest/Mod may not work correctly if you upload it to Steam Workshop.

Compiling GPL source to Bytecode (.bcd)

The GPL bytecode section is special in that you specify the bytecode that is to be loaded, but you can also specify the source files that will be compiled into the bytecode. This eliminates the need to making a separate batch file that loads a GPL project file and compiles the source into bytecode. Once you have specified the target bytecode file and one or more source files that make up that bytecode, you can right click on the Entry and compile the bytecode. The RGSEditor will make a temporary .gplproj file for you in the destination folder and call the GPL compiler. The GPL compiler must be in the same directory as the RGSEditor.exe. This is the

normal case when running the RGSEditor from the SDK. The output from the GPL compiler will be captured in the Log window of the RGSEditor.

You can also right click on the GPL Bytecode entry and select Compile All to compile all of the GPL Entries, if you have more than one. This can also be accomplished by selecting Compile GPL from the Quest menu when editing a Quest or the Mod menu when editing a Mod.

Quest Definition File Format

This section will describe all the entries a Quest Definition recognizes. The master Quest Definition file is an XML formatted file and can be manually edited with Notepad or other text/XML editor, though you should be able to edit all the relevant fields in the RGSEditor's Project View. If you do edit the quest definition file manually, be sure that the file is not also open in the RGS Editor. This precaution will prevent the RGS Editor from possibly overwriting your changes.

This section will use the Wrath Of Krolm Quest, contained in the SDK as an example. The definition for that quest (WrathOfKrolm.mqxml) looks like this:

```
<Majesty>
  <Quest id="{9757F443-4229-41dc-860B-9BD980D72733}">
    <Name>WrathOfKrolm</Name>
    <DisplayName lang="en_US">Wrath of Krolm - The Example</DisplayName>
    <Description lang="en_US">
      <Short>Destroy all Altars to Krolm, or...</Short>
      <Long>"The followers of Krolm have challenged..."</Long>
    </Description>
    <Difficulty>Hard</Difficulty>
    <DataConfiguration>
      <Dataset base="Majesty">
        <Load>
          <Descriptions>Data/WrathOfKrolm_Actions.xml</Descriptions>
          <Descriptions>Data/WrathOfKrolm_Sounds.xml</Descriptions>
          <Descriptions>Data/WrathOfKrolm_Characters.xml</Descriptions>
          <Descriptions>Data/WrathOfKrolm_Buildings.xml</Descriptions>
          <Descriptions>Data/WrathOfKrolm_Overlays.xml</Descriptions>
          <CAM>Data/WrathOfKrolm_maindata.cam</CAM>
          <CAM>Data/WrathOfKrolm_soundfx.cam</CAM>
          <CAM>Data/WrathOfKrolm_voices.cam</CAM>
          <GPL>Data/WrathOfKrolm.bcd</GPL>
          <GPLSource>GPL</GPLSource>
          <Template>Quests/WrathOfKrolm.q</Template>
          <Strings>Data/WrathOfKrolm_Text.xml</Strings>
        </Load>
        <Unload>
          <CAM/>
          <GPL/>
          <Constants/>
          </Unload>
      </Dataset>
    </DataConfiguration>
  </Quest>
</Majesty>
```

The top level tag will always be **Majesty**.

The next XML tag in the hierarchy is **Quest,** which starts the formal Quest Definition. The **Quest** tag has one attribute named **id**. The **id** attribute contains the unique identifier for the Quest. The **id** is a standard Global Unique ID or GUID for short. The RGS Editor will

generate one for the quest when it is created. It is important that the **id** is unique for each quest, do not re-use the **id** for another quest. If you want to use a quest as a starting point for a new quest, re-generate the **id**. When a user is playing a Quest and saves the game, the id will be put in the save game, so that it knows what Quest data to load when the save game is loaded, so it is important to keep the id unique as well as to try and keep it the same as you are developing a Quest. Changing the id will cause any previous save game to become invalid as they will no longer know what Quest to load.

The first child tag of the Quest is **Name**. The **Name** tag is a compact identifier for the Quest, no spaces or special characters are allowed, just letters and numbers. The **Name** tag's primary use is as the name of the GPL function that will be called when the Quest begins. If there is no GPL function of that name, the Quest will still run, but there will be no special initialization or victory conditions.

The next tag in the Quest is **DisplayName**. This tag's contents are used when the name of the Quest is to be displayed on the screen, such as in the Quest list. The **DisplayName** tag has one attribute called **lang**. This attribute contains the language/country code the contents of the **DisplayName** tag are in. The example shows the **DisplayName** using “en_US”, which is the English language version for the US. The allowed language codes are:

"en_US"	Enlgish, USA
"fr_FR"	French, France
"de_DE"	German, Germany
"es_ES"	Spanish, Spain
"it_IT"	Italian, Italy
"en_GB"	English, Great Britain
"ja_JA"	Kanji, Japan
"ko_KR"	Korean, Korea
"zh_CN"	Simplified Chinese, China
"zh_TW"	Simplified Chinese, Taiwan
"pl_PL"	Polish, Poland
"ru_RU"	Russian, Russia
"pt_PT"	Portuguese, Portugal

Multiple **DisplayName** entries are allowed, each with a different **lang** attribute, which allows for the Quest to be translated for different regions. If a user's game requests a language that does not exist, the request will fall back to using the en_US entry.

The next tag is **Description**, which contains child tags **Short** and **Long**. The **Description** tag is for defining a short description, which usually contains the goals of the Quest, and a long description that is displayed at the start of the Quest. The **Short** description is available for review by the player during the game, so it is important that it repeats the goals for the Quest to remind the player what to do. Like the **DisplayName** tag, the **Description** tag has a **lang** attribute, defining the language the **Description** is in. There can also be multiple **Description** tags, defining different language translations.

The next tag is **Difficulty**, which is to tell the user how difficult the Quest is. The **Difficulty** tag can have a value of **Easy**, **Medium**, **Hard** or **Master**.

The next tag is **DataConfiguration**, and it begins the section that defines the files the Quest Definition will load or unload. The child of **DataConfiguration,** named **DataSet,** defines those load and unload commands. The **DataSet** tag also has an attribute called **base**, which signifies what set of files from the Majesty the Quest will start with. The value for the base attribute can be either **Majesty** or **MajestyExpansion**. If *base="Majesty"*, then the Quest will use the art/sound/GPL from the original Majesty game. If *base="MajestyExpansion"*, then the Quest will use the art/sound/GPL from Majesty: The Northern Expansion. It should be noted that once you pick which **base** data set to start from in the RGS Editor, manually changing the base attribute can lead to erratic behavior or crashes as the Quest Template is tightly coupled to the data set as it was defined in the RGS Editor.

The **DataSet** has two child tags, one is called **Load** the other is called **Unload**. They define what new files to load for the Quest, as well as optionally unloading a file. The latter is rare, but some more complicated quests may unload a file that is completely replaced by a new file's contents.

The **Load** and **Unload** child tags define what type of data to load or unload. There can be multiple tags of the same type for each of the data types, except for the Template, which only has one entry.

The first data type tag in the example is **Descriptions**. These are XML files containing the descriptions for new assets the Quest is using, such as Character types, Building types, Sounds and Actions. The contents of these descriptions can also be modifications to existing assets.

The next type in the example is **CAM**, which contain the binary data for assets, primarily art and sound.

The next type is **GPL**, which defines the compiled GPL code the Quest is using. See the *Writing and Compiling GPL Code* section as well as the *GPL Reference* for details.

The type tag **GPLSource** is used to tell the GPL Debugger where the source code for GPL bytecode is located. This is optional to add, but is useful if you are using the GPL Debugger when developing a Quest or Mod. The **GPLSource** tag should contain a relative path from where the Quest definition file is located. In the example it contains the folder name *GPL*, where the source files for the example Quest are located. Multiple **GPLSource** tags can be specified if the you have source in multiple directories, though you do not have to specify sub-folders of a previously defined **GPLSource** path, the debugger will search recursively through the sub-folders.

The next data type is **Template**, which defines the RGS template the Quests is using. The template contains the initial layout of the Quest and is the primary data type that is edited by the RGS Editor.

The last data type is **Text**, which defines any files that contain text that is displayed by the Quest. The layout if the text in the text definition files is described in the *Translatable Quest Text* section.

Mod Definitions

Mod Definitions are similar in form and function to a Quest Definition. A Mod loads additional GPL code, data, etc. when activated, just like a Quest. Mods can be created and edited with the RGSEditor, by selecting New Mod... from the File menu. This will create a new document, where the Mod's values can be edited in the Project View. Mods will not show a map view as they are independent of a Quest map. See **The Project View** for information on editing the project definition.

Mods are different from Quests in that they are loaded along with a Quest, allowing modification to the GPL and data that a Quest sees as its base data set. Mods specify what base data set they work with, either **Majesty**, **MajestyExpansion**, or **Any**. If a mod specifies that it will work with **Any** base data set, the Mod will be loaded with any Quest. Alternately, if a mod specifies that it works with **Majesty** or **MajestyExpansion** data, it will only be loaded if the Quest uses the same base data set. A Mod can specify that it works with **Any** data set if what it modifies is common to both **Majesty** and **MajestyExpansion** or adds completely new data. Multiple Mods can be active at the same time and will load in the order they appear in the Active Mods list. If a Mod is in the Active Mods list, but does not work with the Quest that is being played, it will not be loaded.

The differences in the tags of the Mod definition are that the primary tag is **Mod** rather than **Quest** and that the **Dataset** tag can contain a base data set reference of **Any**. **Template** tags in a Mod definition are ignored. Mod definition use the file extension .mmxml. Below is the sample Mod from the SDK, Adjust Guardhouse.

```
<Majesty>
      <Mod id="{D666FCBE-9906-4582-A6A0-DC96F812441E}">
            <Name>AdjustGuardHouse</Name>
            <DisplayName lang="en_US">Adjust Guardhouse</DisplayName>
            <Description lang="en_US">
                  <Short>Upgraded guard houses increase the number of guards posted
at the guard house to two.</Short>
                  <Long></Long>
            </Description>
            <DataConfiguration>
                  <Dataset base="Any">
                        <Load>
                              <GPL>Data/AdjustGuardhouse.bcd</GPL>
                        </Load>
                  </Dataset>
            </DataConfiguration>
      </Mod>
</Majesty>
```

Data Descriptions

Some of the objects and actions in the game are defined by data descriptions. Majesty shipped with these descriptions in a binary format, however the contents of these binary files are provided in XML form in the SDK and any user generated Quests can create or modify description XML files and add them to the Quest Definition. See *Quest Definition File* Format for more information on how to add files to the Quest Definition.

The object definitions have a few common attributes, such as the type, sub-type, name and ID. The description hierarchy then splits into Engine and Game subsections. The Engine section contains common values that are not game specific, and the Game section contains values related to Majesty.

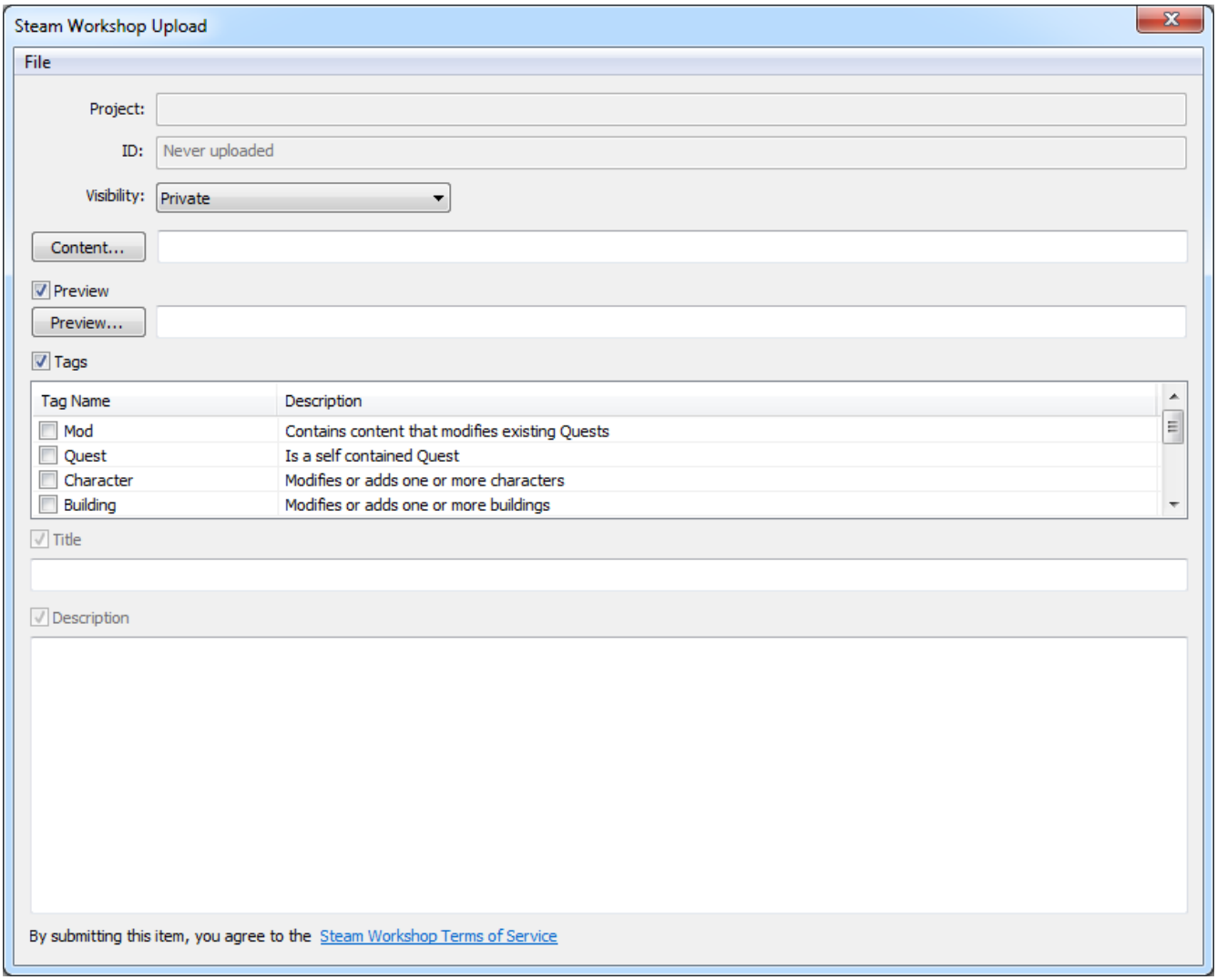
The primary types are Units and Sounds. The Unit type uses the subtype attribute to define whether the unit description is of a Building, Character, Projectile, Overlay or Particle System. The Engine section for each of these sub types is the same, the Game section contains values specific to that sub type.

Steam Workshop

Once your Mod or Quest is complete, you can share it with the rest of the Majesty community through Steam Workshop. Uploading your Mod or Quest is done by selection the Steam Workshop Upload... item in the File menu of the RGSEditor. If the Steam Workshop Upload... item is grayed out, it usually means that the Steam Client is not running or is not able to communicate with the Steam servers. Exit out of the RGSEditor, then start or restart the Steam Client, then restart the RGSEditor.

Setting up a Workshop Project

After selecting Steam Workshop Upload..., you will be presented with the project dialog.



The first value you can change is the **Visibility** of your Workshop item. This controls who can see your item and can be Private, where only you can see it, Friends Only, where only you and your Steam friends can see it and Public, where everyone can see it. You may want to

keep it Private or Friends Only until you are sure your Quest or Mod is stable.

The second value you can change is the **Content** location. This is the directory where the content for your Mod or Quest resides. Usually you will point this to where you have been building and testing your mod locally, such as C:\Users\MyLoginName\Documents\My Games\MajestyHD\Mods\MyCoolMod.

Please note that everything in the content directory is uploaded to Steam Workshop, including all the contents of sub directories in the content folder. Keep the contents as small as possible, supplying just what is contained in the Mod or Quest. This can be the source to the Mod or Quest too, since that is not too big and it is encouraged that you do supply the .gpl files for your Mod or Quest, so other users can view what you changed. It is also encouraged that you make your Mod/Quest so that it is built from just the changes you made to the SDK, and not the whole SDK itself.

The next value to edit is the **Preview** image for your Workshop item. It is not required, but it is encouraged that you take a screen shot or make some artwork that reflects the contents of your Mod/Quest. The image should be either a .png or .jpg and under 1mb in size. The image will be scaled by the Steam backend. Do NOT put the image in your content folder, you should put it in a folder where you save your Workshop project file, which is the file you will save once you complete filling in the Workshop item definition. It is best to save it in a folder that is in the MajestyHD documents directory, just above where your Mod/Quest is save. i.e. C:\Users\MyLoginName\Documents\My Games\MajestyHD\Workshop Projects

The next value to change is the **Tags** that are relevant to your Mod/Quest. These tags help other users search for your Mod/Quest. Check all of the tags you feel describe your Mod/Quest. If your Mod works with both the original Majesty rules/code and the Norther Expansion rules/code, check both of those tags.

The next value to edit is the **Title** of your Mod/Quest. This will be show in a larger font in the Workshop page. You must supply a title for the Mod/Quest to be uploaded and the text should be 128 character or less.

The final value to edit is a **Description** of your Mod/Quest. This should tell the user what the Mod changes or what the Quest's goals are. You must supply a description for the Mod/Quest to be uploaded and the text should be 5000 character or less.

Once you have filled in all the values for the project, you should save the project so you can load it back in if you need to update your Mod/Quest. To save your project, select **Save** or **Save As...** from the File menu that is inside the Upload dialog.

You can now upload your project to Steam Workshop by selecting **Upload...** from the File menu that is inside the Upload dialog. You will be presented with a dialog that will allow you to enter an optional change note that is available for other users to see. If after selecting Upload... you do not see the Upload dialog, it may be because your project does not have valid values in it, go back and re-check that the Content directory is correct and you have a valid Title and Description.

After you have entered the optional Change Note, press **OK** to start the upload. You will be

presented with a progress dialog that will show what is happening with the upload. Please note that you cannot cancel the upload, and if the upload is taking a while, you can continue back to the RGSEditor and the upload will be done in the background. The actual upload is handled by the Steam Client and not the RGSEditor. If you have never upload to Steam Workshop before or the Steam Workshop terms have changed, you will need to accept the Steam Workshop terms before your item can be made public to any other users, including friends.

Important!

Once you have exited the Upload progress dialog, it is very important to save your Project file again if this is the first time you have uploaded your Workshop item. On the first upload, Steam will give your item an ID value that will be saved into the project so that when you re-load the project it will know which project to update. If for some reason you forget to save the item after the ID has been assigned, you can retrieve the Workshop item ID, by going to your Workshop item in either the Steam Client, or web browser and look at the URL when you are on the item's page. It will be at the end of the URL string, ?id=yourItemNumber. You can open the project file with Notepad or other text editor and put that number in the **id** attribute of the **SteamWorkshop** tag.

Updating

If you need to update your Mod/Quest, simply reload your saved Workshop project, and select **Upload...** You will note that when you re-load your Workshop project, most of the values in the project, such as the Preview, Tag, Title and Description will be unchecked and grayed out. When updating, you usually do not need to update those values, and if they are unchecked, the uploader will not re-upload them. This is often desirable if you use the Steam Workshop pages from the Steam Client or web browser to change the values after uploading. The RGSEditor's uploader does not download any changes made from the Workshop item page so if you re-upload any of those value they will overwrite your changes made through the Workshop pages. If you do want to upload those value, check the value's title and the value will be un-grayed and will upload, if changed.

Subscribing To And Loading A Mod/Quest

Once you have uploaded your item, users, including yourself, can go to the Workshop page in the Steam Client or web browser and subscribe to your item. The Steam Client will then download the Workshop item to a directory where Majesty can find it. Please note this is NOT the same location where Majesty scans for your locally created Mod/Quests in your MyDocuments folder. The download location is inside the Steam install directory. If you subscribe to your own Mod/Quest, you will see it listed twice in the list in Majesty. This first one will always be your local copy, the second will be the Steam subscribed copy. You can activate the Steam copy, but be aware that if you load a save game that is using the Mod/Quest, the loader looks for a matching Mod/Quest ID and will always pick the one from the My Documents directory first.